

torch.jit.trace#

<https://pytorch.org/docs/stable/generated/torch.jit.trace.html#torch.jit.trace>

Description:

Trace a function and return an executable or ScriptFunction that will be optimized using just-in-time compilation. Tracing is ideal for code that operates only on Tensor\ and lists, dictionaries, and tuples of Tensor\ . Using torch.jit.trace and torch.jit.trace_module, you can turn an existing module or Python function into a TorchScript ScriptFunction or ScriptModule. You must provide example inputs, and we run the function, recording the operations performed on all the tensors. The resulting recording of a standalone function produces ScriptFunction. The resulting recording of nn.Module.forward or nn.Module produces ScriptModule.

Function Signature

```
torch.jit.trace(func, example_inputs=None, optimize=None,
check_trace=True, check_inputs=None, check_tolerance=1e-05,
strict=True, _force_outplace=False, _module_class=None,
_compilation_unit=<torch.jit.CompilationUnit object>,
example_kwarg_inputs=None, _store_inputs=True)[source]#
```

Parameters

Keyword Arguments

Returns

Parameters

Keyword

`example_inputs` (tuple or `torch.Tensor` or `None`, optional) – A tuple of example inputs that will be passed to the function while tracing. Default: `None`. Either this argument or `example_kwarg_inputs` should be specified. The resulting trace can be run with inputs of different types and shapes assuming the traced operations support those types and shapes. `example_inputs` may also be a single `Tensor` in which case it is automatically wrapped in a tuple. When the value is `None`, `example_kwarg_inputs` should be specified. `check_trace` (bool, optional) – Check if the same inputs run through traced code produce the same outputs. Default: `True`. You might want to disable this if, for example, your network contains non-deterministic ops or if you are sure that the network is correct despite a checker failure.

Returns:

If `func` is `nn.Module` or `forward` of `nn.Module`, `trace` returns a `ScriptModule` object with a single `forward` method containing the traced code. The returned `ScriptModule` will have the same set of sub-modules and parameters as the original `nn.Module`. If `func` is a standalone function, `trace` returns `ScriptFunction`.

Code Examples

```
import torch

def foo(x, y):
    return 2 * x + y

# Run `foo` with the provided inputs and record the tensor
# operations
traced_foo = torch.jit.trace(foo, (torch.rand(3),
torch.rand(3)))

# `traced_foo` can now be run with the TorchScript interpreter
# or saved
# and loaded in a Python-free environment
```

```
import torch
import torch.nn as nn

class Net(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv = nn.Conv2d(1, 1, 3)

    def forward(self, x):
        return self.conv(x)

n = Net()
example_weight = torch.rand(1, 1, 3, 3)
example_forward_input = torch.rand(1, 1, 3, 3)

# Trace a specific method and construct `ScriptModule` with
# a single `forward` method
module = torch.jit.trace(n.forward, example_forward_input)

# Trace a module (implicitly traces `forward`) and construct a
# `ScriptModule` with a single `forward` method
module = torch.jit.trace(n, example_forward_input)
```

Notes & Warnings

WARNING

Warning Tracing only correctly records functions and modules which are not data dependent (e.g., do not have conditionals on data in tensors) and do not have any untracked external dependencies (e.g., perform input/output or access global variables). Tracing only records operations done when the given function is run on the given tensors. Therefore, the returned ScriptModule will always run the same traced graph on any input. This has some important implications when your module is expected to run different sets of operations, depending on the input and/or the module state. For example, Tracing will not record any control-flow like if-statements or loops. When this control-flow is constant across your module, this is fine and it often inlines the control-flow decisions. But sometimes the

Detailed Documentation

Documentation

Trace a function and return an executable or ScriptFunction that will be optimized using just-in-time compilation.

Tracing is ideal for code that operates only on Tensor\`s and lists, dictionaries, and tuples of Tensor\`s.

Using `torch.jit.trace` and `torch.jit.trace_module`, you can turn an existing module or

Python function into a TorchScript `ScriptFunction` or `ScriptModule`. You must provide example inputs, and we run the function, recording the operations performed on all the tensors.

The resulting recording of a standalone function produces `ScriptFunction`.

The resulting recording of `nn.Module.forward` or `nn.Module` produces `ScriptModule`.

This module also contains any parameters that the original module had as well.

Tracing only correctly records functions and modules which are not data dependent (e.g., do not have conditionals on data in tensors) and do not have any untracked external dependencies (e.g., perform input/output or access global variables). Tracing only records operations done when the given function is run on the given tensors. Therefore, the returned `ScriptModule` will always run the same traced graph on any input. This has some important implications when your module is expected to run different sets of operations, depending on the input and/or the module state. For example,

Tracing will not record any control-flow like if-statements or loops. When this control-flow is constant across your module, this is fine and it often inlines the control-flow decisions. But sometimes the control-flow is actually part of the model itself. For instance, a recurrent network is a loop over the (possibly dynamic) length of an input sequence.

In the returned `ScriptModule`, operations that have different behaviors in training and eval modes will always behave as if it is in the mode it was in during tracing, no matter which mode the `ScriptModule` is in.

In cases like these, tracing would not be appropriate and scripting is a better choice. If you trace such models, you may silently get incorrect results on subsequent invocations of the model. The tracer will try to emit warnings when doing something

that may cause an incorrect trace to be produced.

`func` (callable or `torch.nn.Module`) – A Python function or `torch.nn.Module` that will be run with `example_inputs`. `func` arguments and return values must be tensors or (possibly nested) tuples that contain tensors. When a module is passed `torch.jit.trace`, only the forward method is run and traced (see `torch.jit.trace` for details).

`example_inputs` (tuple or `torch.Tensor` or `None`, optional) – A tuple of example inputs that will be passed to the function while tracing. Default: `None`. Either this argument or `example_kwarg_inputs` should be specified. The resulting trace can be run with inputs of different types and shapes assuming the traced operations support those types and shapes. `example_inputs` may also be a single `Tensor` in which case it is automatically wrapped in a tuple. When the value is `None`, `example_kwarg_inputs` should be specified.

`check_trace` (bool, optional) – Check if the same inputs run through traced code produce the same outputs. Default: `True`. You might want to disable this if, for example, your network contains non-deterministic ops or if you are sure that the network is correct despite a checker failure.

`check_inputs` (list of tuples, optional) – A list of tuples of input arguments that should be used to check the trace against what is expected. Each tuple is equivalent to a set of input arguments that would be specified in `example_inputs`. For best results, pass in a set of checking inputs representative of the space of shapes and types of inputs you expect the network to see. If not specified, the original `example_inputs` are used for checking

`check_tolerance` (float, optional) – Floating-point comparison tolerance to use in the checker procedure. This can be used to relax the checker strictness in the event that results diverge numerically for a known reason, such as operator fusion.

`strict` (bool, optional) – run the tracer in a strict mode or not (default: True). Only turn this off when you want the tracer to record your mutable container types (currently list/dict) and you are sure that the container you are using in your problem is a constant structure and does not get used as control flow (if, for) conditions.

`example_kwarg_inputs` (dict, optional) – This parameter is a pack of keyword arguments of example inputs that will be passed to the function while tracing. Default: None. Either this argument or `example_inputs` should be specified. The dict will be unpacking by the arguments name of the traced function. If the keys of the dict don't match with the traced function's arguments name, a runtime exception will be raised.

If `func` is `nn.Module` or forward of `nn.Module`, `trace` returns a `ScriptModule` object with a single forward method containing the traced code. The returned `ScriptModule` will have the same set of sub-modules and parameters as the original `nn.Module`. If `func` is a standalone function, `trace` returns `ScriptFunction`.

Example (tracing a function):

Example (tracing an existing module):